

Extending RISC-V to Support Flexible-Radix Multiply-Accumulate Operations

Isaar Ahmad¹ , Hao Cheng^{2,3,4} , Johann Großschädl⁵  and Daniel Page¹ 

¹ University of Bristol, School of Computer Science, Bristol, UK

² Shandong University, School of Cyber Science and Technology, Qingdao, China

³ Shandong University, State Key Laboratory of Cryptography and Digital Economy Security, Qingdao, China

⁴ Shandong University, Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Qingdao, China

⁵ University of Luxembourg, DCS and SnT, Esch-sur-Alzette, Luxembourg

Abstract. Specified as part of the (standard, optional) M extension, the `mul` and `mulhu` instructions reflect support for unsigned integer multiplication in RISC-V base Instruction Set Architectures (ISAs) such as RV32I and RV64I: given w -bit integers x and y for a word size w , they respectively produce the less- and more-significant w bits of the $(2 \cdot w)$ -bit product $r = x \times y$. This typically minimal, and hence RISC-like form contrasts sharply with many alternative ISAs. For example, ARMv7-M includes a rich set of multiply and multiply-accumulate instructions; these cater for a wide variety of important use-cases in cryptography, where multi-precision integer arithmetic is often a central requirement. In this paper, we explore the extension of RV32I and RV64I, i.e., an Instruction Set Extension (ISE), with richer support for unsigned integer multiplication. Our design has three central features: 1) it includes dedicated carry propagation and multiply-accumulate instructions, 2) those instructions allow flexible selection of the radix (thus catering for reduced- and full-radix representations), and 3) the design can be considered for any w , and so uniformly across both RV32I and RV64I. A headline outcome of our evaluation is that, for X25519-based scalar multiplication, use of the ISE affords $1.5\times$ and $1.6\times$ improvement for full- and reduced-radix cases, respectively, on RV32I, and $1.3\times$ and $1.7\times$ improvement for full- and reduced-radix cases, respectively, on RV64I.

Keywords: RISC-V · ISA · ISE · multi-precision integer arithmetic

1 Introduction

Many important public-key cryptosystems operate in widely-studied algebraic structures like groups, rings, or fields, whose elements are integers of a length of hundreds, or even thousands, of bits. Examples include not only classical schemes such as RSA, DSA, and Diffie-Hellman [Gal12], as well as elliptic-curve variants of the latter (i.e., ECDSA and ECDH [HMOV04]), but also some more recent cryptosystems based on bilinear pairings defined over groups [EJ17] or isogenies between supersingular curves [?]. All these cryptosystems have in common that, at the lowest level, they perform arithmetic operations (i.e., addition, subtraction, multiplication, squaring, inversion) of large integers modulo a prime or a product of primes or a near-prime integer [Gal12]. Most software

E-mail: isaar.ahmad@bristol.ac.uk (Isaar Ahmad), hao.cheng@sdu.edu.cn (Hao Cheng), johann.groszschadl@uni.lu (Johann Großschädl), daniel.page@bristol.ac.uk (Daniel Page)



libraries for Multi-Precision Integer (MPI) arithmetic represent the operands (which are integers of a length of n bits) via arrays of w -bit *digits* or *limbs*, where w equals the word-size of the underlying processor (called “full-radix” representation) or is a few bits below the word-size (“reduced-radix” representation). In general, a radix- 2^w representation splits an n -bit integer A into $l = \lceil n/w \rceil$ digits/limbs so that $A = \sum_{i=0}^{l-1} a_i 2^{iw}$ where $0 \leq a_i < 2^w$ for $0 \leq i < l$. The most common word-sizes of general-purpose processors (resp. microcontrollers) are 8, 16, 32 and 64 bits. Algorithms for MPI arithmetic operate on the digits or limbs of such arrays by executing w -bit instructions supported by the processor, e.g., w -bit addition or $(w \times w)$ -bit multiplication. The most basic techniques for MPI multiplication, namely the so-called *operand-scanning* method [MOV96] and *product-scanning* method [Com90a], have complexity $\mathcal{O}(l^2)$ for l -digit operands, i.e., the number of $(w \times w)$ -bit multiply instructions increases with the square of the operand length. Besides the length of the operands, the execution time of an MPI multiplication is also influenced by certain features of the Instruction Set Architecture (ISA), e.g., the actual semantics of the $(w \times w)$ -bit multiplication and some other instructions, and the micro-architecture, e.g., the latency of the $(w \times w)$ -bit multiply instruction.

The $(w \times w)$ -bit multiply instruction is “special” in the sense that its result (i.e., the product of two w -bit digits or limbs) has a length of $2w$ bits and does, hence, not fit into a single register. This contrasts with most other arithmetic/logical instructions, such as instructions for Boolean operations like AND, OR, and XOR, which follow (at least in the case of RISC ISAs) the common 3-register instruction format, i.e., two source registers and one destination register. Over time, computer architects have come up with a broad and highly diverse range of approaches on how to deal with the “widening” aspect of the multiply instruction. For example, the multiply instructions of the 8-bit AVR and the MIPS32 architecture allows one to specify only the two source registers; the product is placed in either two fixed general-purpose registers (e.g., `r0` and `r1` in AVR) or special registers (e.g., `hi` and `lo` in MIPS32). In the latter case, the ISA provides instructions (e.g., `mvlo`, `mvhi`) that allow one to transfer the content of these two special registers to a general-purpose register. The SPARC V8 architecture implements a mix of these two approaches since its multiply instruction puts the lower half of the $2w$ -bit product into a general-purpose register and upper half in a special register named `%y`. There are also some ISAs that offer separate multiply instructions for the lower and upper half of the product; well-known examples are the PowerPC architecture and RISC-V [?]. The ARM ISA provides the most flexible support for “widening” multiplication and includes also some other instructions that are useful for MPI arithmetic. For example, ARMv7-M contains the `umull` instruction for integer multiplication, which allows a programmer to specify two general-purpose registers for storing the 64-bit product, see [ARM21a, Section A4.4.3]. However, this flexibility comes at a price, namely when single-cycle execution is desired (as is the case for Cortex-M4 microcontrollers), a total of four buses between the register file and the multiplier are needed and the register file has to provide a second write port (i.e., two read ports and two write ports altogether).

ARMv7-M [ARM21a, Section A4.4.3] offers a concrete example, in the sense it offers a rich set of instructions for unsigned integer multiplication that includes `mul` (multiply), `m1a` (multiply-accumulate), `umull` (multiply), `umlal` (multiply-accumulate long), and `umaal` (multiply-accumulate-accumulate long). These cases use up to 4 source and destination operands (with the former reused for the latter in some cases), to cater for 1) different computation (e.g., signed versus unsigned, including or excluding accumulation), and 2) different types (e.g., a half- or full-width product). In contrast, RISC-V [RIS24a], without loss of generality RV32I [RIS24a, Section 2] meaning $w = 32$, does not include *any* support for multiplication by default. Instead, such support is provided by the standard `M1` (or

¹The `M` extension actually includes support for multiplication *and* division; `Zmmul` acts as a multiplication-only subset of `M`, for contexts (e.g., micro-controllers) in which support for the former alone

multiplication) extension [RIS24a, Section 13] which includes `mul` (multiply) and `mulhu` (multiply-high). These instructions allow computation of the least- and most-significant 32 bits of some 64-bit, unsigned product, so, if used as a pair, are analogous to one ARMv7-M `umull` instruction; no analogous support for multiply-accumulate exists.

Contributions. In this paper, we start with the premise that software-based implementation of multi-precision integer arithmetic may, depending on the context, either be advantageous or necessary. Based on this, our focus is a (non-standard, i.e., custom) extension of the RISC-V base Instruction Set Architecture (ISA) to more effectively support such implementations. Noting that associated material (e.g., source code relating to hardware and software implementations) is available² under an open source license, we make the following contributions:

1. We design, implement, and evaluate an Instruction Set Extension (ISE) which has 3 central features: 1) it includes dedicated carry propagation and multiply-accumulate instructions, 2) those instructions allow flexible selection of the radix (thus catering for reduced- *and* full-radix representations), and 3) the design can be considered for any w , and so uniformly across both RV32I [RIS24a, Section 2] (where $w = 32$) and RV64I [RIS24a, Section 4] (where $w = 64$). Doing so can therefore be viewed as a generalisation of work by Cheng et al. [CFG⁺24], with respect to features 2) *and* 3): [CFG⁺24] describes an *algorithm*-specific ISE, i.e., specialised to support one construction via a fixed selection of radix, whereas this paper describes a *domain*-specific ISE, i.e., generalised to support a domain of constructions via flexible selection of the ideal radix for each instance. As further illustrated by Table 1, feature 2) is also novel with respect to existing base ISAs.
2. We present an evaluation based on both lower- and higher-level arithmetic use-cases. With respect to the former, we use the ISE to explore the relative merits of reduced- versus full-radix representations for multi-precision integer multiplication. Doing so can therefore be viewed as an extension of work by Stoffelen [Sto19], who, for the specific, RV32I-based evaluation platform used, concludes that “[*s*]choolbook multiplication with reduced-radix representations is only 8% faster than multiplication with non-reduced representations”. With respect to the latter, we present optimised ISA- and ISE-based implementations of X25519 [Ber06] for both RV32I and RV64I.

We stress that an underlying goal is to simultaneously deliver efficiency while *also* maximising flexibility and thus (cryptographic) agility: NIST [Nat25] provide a detailed explanation of and motivation for the latter. For example, given an n one might maximise b in order to minimise l but simultaneously constrain it to permit compatibility with 1) algorithmic (e.g., permitting a large enough accumulator that overflow does not occur in a product-scanning integer multiplication) and/or 2) optimisation (e.g., efficient modular reduction, delayed reduction, and so on) techniques. For a word size of $w = 64$, for example, implementations of Curve25519 (see [Ber06]) often select $b = 51$ (for $n = 255$, meaning $l = 5$) to facilitate efficient modular reduction; in contrast, implementations of Curve448 (see [Ham15]) often select $b = 56$ (for $n = 448$, meaning $l = 8$). Allowing flexible selection of b allows optimisation to the specific implementation *and* parameterisation of *any* construction, rather than fixing a “one size fits all” selection. As well as maximising efficiency, doing so arguably also delivers generality and so maximises utility of the support provided; a micro-architecture which implements the ISE will remain useful in the event a construction is changed (e.g., due to a cryptanalytic result: cf. [CD23, MMP⁺23, Rob23] in relation to SIKE [JAC⁺20]) post-deployment, for example.

is useful.

²<https://github.com/bristolcrypto/mpise>, <https://github.com/bristolcrypto/mpise-cva6>

Table 1: An overview of support for unsigned multiply(-accumulate) instructions across base ISAs and ISEs. Note that $\dagger$ (resp. $\ddagger$) denotes a requirement for a standard (resp. non-standard, i.e., custom) extension to the cited base ISA; $\bigcirc$ denotes 1-instruction half-product (i.e., least-significant w bits of full $(2 \cdot w)$ -bit product), $\bullet$ denotes 1-instruction full-product (i.e., full $(2 \cdot w)$ -bit product), and $\bullet$ denotes 2-instruction full-product.

Reference	Base ISA	Support			
		Multiply		Multiply-accumulate	Flexible radix
[ARM18]	ARMv6-M	✓	$\bigcirc$	×	×
[ARM21a]	ARMv7-M	✓	$\bullet$	✓ $\bullet$	×
[ARM21b]	ARMv8-M	✓	$\bullet$	✓ $\bullet$	×
[RIS24a]	RV32I	✓ $\dagger$	$\bullet$	×	×
	RV64I	✓ $\dagger$	$\bullet$	×	×
[CFG ⁺ 24]	RV64I	✓ $\ddagger$	$\bullet$	✓ $\ddagger$ $\bullet$	×
this work	RV32I	✓ $\ddagger$	$\bullet$	✓ $\ddagger$ $\bullet$	✓ $\ddagger$
	RV64I	✓ $\ddagger$	$\bullet$	✓ $\ddagger$ $\bullet$	✓ $\ddagger$

2 Background

2.1 Notation

Let $x_{(b)}$ denote x expressed in radix- or base- b . If the base is omitted, it is safe to assume use of decimal (i.e., that $b = 10$). Let $x \leftarrow y$ denote assignment of y to x . Let $\text{ext}_w^\pm(x)$ (resp. $\text{ext}_w^0(x)$) denote sign-extension (resp. zero-extension) of some x to w bits. Let $\text{LSB}_x(y)$ (resp. $\text{MSB}_x(y)$) denote the least-significant (resp. most-significant) x bits of y . Let $\neg$, $\wedge$, $\vee$, and $\oplus$ denote the Boolean NOT, AND, (inclusive) OR, and (exclusive OR, or) XOR operators respectively, and $x \ll y$ and $x \lll y$ (resp. $x \gg y$ and $x \ggg y$) denote left-shift and left-rotate (resp. right-shift and right-rotate) of x by y bits.

Let $\text{MEM}[i]^b$ denote a b -byte access to some byte-addressable memory, using the effective address i ; where $b = 1$, the access granularity may be omitted. Let $\text{GPR}[i]$, for $0 \leq i < e$, denote the i -th, w -bit entry in an e -entry general-purpose register file.

2.2 RISC-V

Although *still* a (relatively) new ISA, roughly 15 years after the RISC-V project was initiated (see context offered, e.g., by Waterman [Wat16, Chapter 1]) it seems clear that it has impacted both industry and academia in a significant way. In both cases, two features in particular have acted to foster innovation spanning both hardware and software. First, it adopts strongly RISC-oriented design principles but is highly modular: a sparse, general-purpose base ISA, can be augmented with special-purpose (or even domain-specific), standard and non-standard extensions. Second, the more open nature (cf. [AP14, HCP⁺16]) of the ISA has (arguably) led to more accessible infrastructure to use as a starting point for further development: open-source base cores and development tool-chains are readily available, for example.

We focus on RV32I and RV64I as the base ISAs for our ISE design, but, due to the CVA6 base core used in Section 4 and Section 5, consider the standard super-sets³ RV32G and RV64G during implementation and evaluation. RISC-V uses XLEN to denote the word size, therefore meaning $w = \text{XLEN} = 32$ or $w = \text{XLEN} = 64$ where the context is RV32I or RV64I respectively. Both RV32I and RV64I specify a 32-element general-purpose register file, in which $\text{GPR}[0] \equiv \text{x0} \equiv \text{zero}$ is fixed to 0 (in the sense reads from it always

³Per [RIS24a, Section 34], the G extension is an abbreviation for “a combination of a base ISA (RV32I or RV64I) plus selected standard extensions (IMAFD, Zicsr, Zifencei)” meaning that RV32G expands to RV32IMAFDZicsr_Zifencei and RV64G expands to RV64IMAFDZicsr_Zifencei.

yield 0 and writes to it are ignored). The “minimal” nature of the RV32I and RV64I base ISAs implies two specific facts that are relevant to this paper:

1. They do not include support for multiplication by default. Instead, and as already outlined in Section 1, the `mul rd, rs1, rs2` $\mapsto$ $\text{GPR}[\text{rd}] \leftarrow \text{LSB}_w(\text{GPR}[\text{rs1}] \times \text{GPR}[\text{rs2}])$ and `mulhu rd, rs1, rs2` $\mapsto$ $\text{GPR}[\text{rd}] \leftarrow \text{MSB}_w(\text{GPR}[\text{rs1}] \times \text{GPR}[\text{rs2}])$ instructions are provided by the standard M (or multiplication) extension [RIS24a, Section 13]. Note that both `mul` and `mulhu` are in the instruction subset specified by Zkt [RIS24a, Section 32.6], and hence must be implemented so they exhibit data-independent execution latency iff. compliance with Zkt is required.
2. They do not include support for carry or overflow management via a status register. Instead, the carry stemming from, e.g., an unsigned addition, is computed via an idiom [PW17, Section 2.6] such as `add a0, a2, a4 ; sltu a2, a0, a2`. The rationale for this choice is that doing so would “*add extra state that is implicitly set by most instructions, which needlessly complicate the dependence calculation for out-of-order execution*” [PW17, Section 2.6], suggesting a higher penalty for lower-end, area-optimised cores than higher-end, throughput-optimised cores which can be more efficiently pipelined and employ fusion of `add` and `sltu`.

2.3 Multi-precision integer arithmetic

As explained in the previous section, Multi-Precision Integer (MPI) arithmetic refers to performing numerical calculations on integers whose length exceeds the word-size of the target hardware architecture. In the case of public-key cryptography, where MPIs can have a length of several hundreds to thousands of bits, the mismatch between the operand-length and processor’s word size (in our case either 32 or 64 bits) can be in the range of one to two orders of magnitude. The concrete implementation of MPI arithmetic requires a software developer to (i) find a suitable representation of the MPIs for efficient storage and processing, and (ii) develop algorithms for performing common arithmetic operations (e.g., addition, multiplication, modular reduction) on MPIs using the fixed-length (i.e., word-size) instructions provided by the target processor.

Representation of operands. Most optimized software libraries represent MPIs through a sequence of digits (i.e., an array of type `uint32_t` or `uint64_t` in C), where the bit-length b of a digit matches the processor’s word size or is slightly below the word size. Such a representation is called *base- 2^b* or *radix- 2^b* representation and formally defined as follows:

Definition 1. A base- or radix- 2^b representation of some n -bit $x \in \mathbb{Z}$ is a sequence of digits $\hat{x} = \langle \hat{x}_0, \hat{x}_1, \dots, \hat{x}_{l-1} \rangle$, such that $x = \sum_{i=0}^{l-1} \hat{x}_i \cdot 2^{b \cdot i}$ and $\hat{x}_i \in \{0, 1, \dots, 2^b - 1\}$.

When b equals the word-size w , the digits are usually referred to as *words*, whereas when b is below w , it is common practice to call them *limbs*. The corresponding MPI representations are termed in the literature as full-radix (i.e., saturated) or reduced-radix (i.e., unsaturated), formally defined as follows:

Definition 2. The term full-radix (resp. reduced-radix) captures the case where, for a base- or radix- 2^b representation of some n -bit $x \in \mathbb{Z}$, $b = w$ (resp. $b < w$), i.e., b is equal to (resp., less-than) the word size.

The full-radix representation is always ideal in terms of storage requirements. On the other hand, when using a reduced-radix representation, the number of limbs may (and usually does) exceed the number of words, especially when the operands are elements of the prime fields of standardized elliptic curve cryptosystems. Historically, it was common practice to choose prime fields such that the bitlength of the prime is a multiple of 32 bits;

```

1 Algorithm 1. MUL-OSM( $\hat{a}, \hat{b}$ ) begin
2   for  $i \leftarrow 0$  to  $l - 1$  do
3      $\hat{z}_i \leftarrow 0$ 
4   end
5   for  $i \leftarrow 0$  to  $l - 1$  do
6      $t \leftarrow 0$ 
7     for  $j \leftarrow 0$  to  $l - 1$  do
8        $s \leftarrow \hat{a}_j \cdot \hat{b}_i + t + \hat{z}_{i+j}$ 
9        $\hat{z}_{i+j} \leftarrow s \bmod 2^b$ 
10       $t \leftarrow s / 2^b$ 
11    end
12     $\hat{z}_{i+l} \leftarrow t$ 
13  end
14  return  $\hat{z}$ 
15 end

```

```

1 Algorithm 2. MUL-PSM( $\hat{a}, \hat{b}$ ) begin
2    $s \leftarrow 0$ 
3   for  $i \leftarrow 0$  to  $l - 1$  do
4     for  $j \leftarrow 0$  to  $i$  do
5        $s \leftarrow s + \hat{a}_j \cdot \hat{b}_{i-j}$ 
6     end
7      $\hat{z}_i \leftarrow s \bmod 2^b$ 
8      $s \leftarrow s / 2^b$ 
9   end
10  for  $i \leftarrow l$  to  $2(l - 1)$  do
11    for  $j \leftarrow i - (l - 1)$  to  $l - 1$  do
12       $s \leftarrow s + \hat{a}_j \cdot \hat{b}_{i-j}$ 
13    end
14     $\hat{z}_i \leftarrow s \bmod 2^b$ 
15     $s \leftarrow s / 2^b$ 
16  end
17   $\hat{z}_{2l-1} \leftarrow s$ 
18  return  $\hat{z}$ 
19 end

```

the most common examples are four of the five so-called NIST primes [Nat99], which have a bitlength of 192, 224, 256, and 384 bits. A reduced-radix representation of a 256-bit integer using a radix of 2^{29} would result in nine limbs (with the most-significant limb containing only 24 bits), whereas a full-radix implementation only requires eight 32-bit words. This difference between the number of limbs and the number of words impacts not only storage cost, but also the number of loop-iterations of common algorithms for MPI arithmetic, which will be covered in more detail below.

There exist also cryptosystems that have been specifically designed with having reduced-radix representations in mind. The most well-known (and, to date, probably still the most widely-used one) is not a public-key scheme but Bernstein’s Poly1305 [NL15, Sect. 2.5], a symmetric authenticator (i.e., message authentication code). As the name indicates, Poly1305 involves multiplications modulo the 130-bit prime $p = 2^{130} - 5$. When targeting 32-bit platforms (see, e.g., [Vai17]), a reduced-radix representation of 130-bit integers using radix 2^{26} consists of five limbs, which is exactly the number of 32-bit words of a full-radix representation. Two examples in ECC are the prime fields over which the elliptic curves Ed3363 (i.e., $p = 2^{336} - 3$ [Sco15]) and Curve2065 (i.e., $p = 2^{206} - 5$ [FGR24]) are defined. In the latter case, a reduced-radix representation with $b = 30$ consists of exactly the same number of limbs as the number of words for a full-radix representation for $w = 32$.

Basic algorithms for MPI multiplication. Besides the representation of operands, MPI arithmetic requires algorithms that are capable of performing operations like (modular) addition, subtraction, multiplication, squaring, and inversion on multi-precision operands using the single-precision (i.e., word-size) instructions provided by the target processor. Of particular interest are (modular) multiplication and squaring, since they have a significant impact on the overall execution time of many public-key cryptosystems. From a high-level point of view, there are two basic algorithms for multi-precision multiplication, namely the *Operand-Scanning Method (OSM)* [MOV96] and the *Product-Scanning Method (PSM)* [Com90b], formally described in Algorithm 1 and Algorithm 2, respectively. Both have in common that they need to execute l^2 single-precision (i.e., $(w \times w \rightarrow 2w)$ -bit) multiplications to obtain the product of two operands consisting of l words/limbs, but they differ regarding their loop structure and inner-loop operations.

The OSM has a simple nested-loop structure and produces the result in a “row-wise”

fashion. In each iteration of the inner loop, an operation of the form $s \leftarrow a \cdot b + t + z$, i.e., two words are multiplied, yielding a $(2 \cdot w)$ -bit product, and two further words are added to it. When using a full-radix representation, each word has a length of $b = w$ bits, and the result of a multiply-add operation of the form $s \leftarrow a \cdot b + t + z$ is at most $2 \cdot b$ bits long. Consequently, the result fits into two registers without overflow. Therefore, a reduced-radix representation would not have any advantage over a full-radix representation and is, therefore, rarely used for the OSM.

The PSM computes the product in a “column-wise” fashion and consists of two nested loops; the first produces the l lower words/limbs of the result and the second the l upper words/limbs. In each iteration of the inner loops, a classical multiply-accumulate operation of the form $s \leftarrow s + a \cdot b$ is carried out, i.e., two words/limbs are multiplied and the product is added to a cumulative sum s . When several such $(2 \cdot b)$ -bit products are accumulated, the sum s will exceed $2 \cdot b$ bits, which means it has to be stored in three registers when a full-radix representation is used. On the other hand, two accumulator registers may still suffice for a reduced-radix representation, but one has to take care to not overflow the cumulative sum s . Concretely, the maximum number of $(2 \cdot b)$ -bit limb-products that can be accumulated into a double-register accumulator is $2^{2 \cdot (w-b)}$.

Apart from the loop structure and inner-loop operation, the OSM and PSM also differ regarding the number of memory accesses performed in the inner loop. A conventional (i.e., not unrolled) implementation of the inner loop of the OSM loads $\hat{a}_j$ and $\hat{z}_{i+j}$ from memory (see Algorithm 1) and writes $\hat{z}_{i+j}$ back, i.e., two loads and one store operation are carried out. The PSM, on the other hand, also performs two loads in the inner loop (for $\hat{a}_j$ and $\hat{b}_{i-j}$, see Algorithm 2), but no store. It is possible to further reduce the number of load operations by taking advantage of the so-called *hybrid multiplication technique* introduced in [GPW⁺04]. The basic idea is to load (and process) more than one word/limb of the two operands in each iteration of the inner loop, thereby reducing not only the loop overhead but also the number of loads. More concretely, when $d > 1$ words/limbs of a and b are loaded in each iteration of the inner loop (i.e., $2 \cdot d$ loads per iteration), the number of inner-loop iterations is reduced by a factor of d^2 . For example, when $d = 4$, eight words/limbs are loaded in each iteration, but the number of iterations is reduced by a factor of 16, which, in turn, halves the number of loads compared to a straightforward implementation of the PSM.

2.4 X25519

X25519 [Ber06] is an efficient Elliptic Curve Diffie-Hellman (ECDH) key exchange scheme that uses a Montgomery curve (called Curve25519) defined over a 255-bit prime field. Montgomery curves allow for a remarkably simple method to compute a variable-base scalar multiplication $Q = kP$, the *Montgomery ladder*, which gets the (affine) x -coordinate of a point P as input and returns the x coordinate of Q . The core operation is a *ladder step* consisting of five multiplications, four squarings, a multiplication by a small constant, and nine additions/subtractions in the underlying prime field $\mathbb{F}_p$. Performing a scalar multiplication according to the Montgomery ladder is not only extremely fast, but also provides some intrinsic protection against timing attacks since it executes always the same sequence of field operations, independent of the actual value of the scalar k .

The prime p used by Curve25519 is a so-called *pseudo-Mersenne prime*, i.e., a prime of the form $p = 2^n - c$ where c is small compared to 2^n (ideally, c fits into a single register of the target platform). More concretely, Curve25519 is defined over the pseudo-Mersenne prime $p = 2^{255} - 19$, which means the elements of $\mathbb{F}_p$ are integers of a length of up to 255 bits. Pseudo-Mersenne primes allow for efficient modular reduction by taking advantage of the congruence $2^n \equiv c \pmod{p}$. In this way, a $2n$ -bit product can be reduced modulo p by multiplying the higher half (i.e., the upper n bits) of the product by c and adding the result to the lower half. Repeating this step a second time yields a (potentially not fully)

reduced result in the range of $[0, 2(p-1)]$, which fits in eight 32-bit (resp., four 64-bit) words when using a full-radix representation or nine 29-bit (resp., five 51-bit) limbs in the reduced-radix case.

2.5 Related work

ISA-only, i.e., pure software, implementation. Stoffelen [Sto19] analyzed the performance of MPI addition and MPI multiplication on the RISC-V architecture (specifically RV32IM) for both full-radix and reduced-radix representation. He also analyzed potential speed-ups enabled by the provision of an “add-with-carry” instruction. Optimized implementations of X25519 exist for both 32-bit and 64-bit RISC-V architectures. For example, van den Berg [vdB20] presents two distinct X25519 implementations for 32-bit RISC-V, with one utilizing the M extension and the other not. Specifically, in the RV32IM version, the refined Karatsuba multiplication [HS15] is used for the prime-field multiplication, and the variable-base scalar multiplication on Curve25519 has an execution time of 5.4M cycles on a SiFive FE310 micro-controller. Erbsen et al. [EPJ⁺24] present a faster RV32IM implementation generated by the Fiat Cryptography framework [EPG⁺19] and Rupicola code generator [PPJ⁺22], where a variable-base scalar multiplication takes 4.1M cycles on the same micro-controller. For 64-bit RISC-V, Gao et al. [GSD⁺24] introduced an optimized assembly implementation, which they benchmarked on a VisionFive board (RV64GC, in-order execution). Scott describes in [Sco24] a highly-optimized C implementation of X25519 for both 32-bit and 64-bit architectures, and reports the execution time of the latter on a LicheePi 4A processor (RV64GC, out-of-order execution).

ISE design and ISE-supported implementation. A significant body of literature deals with the use of ISEs to support multi-precision integer arithmetic: we refer to the Systematization of Knowledge (SoK) presented by Cheng et al. [CGM⁺24] for a comprehensive, general coverage. An important instance of more specific, directly related work is that of Cheng et al. [CFG⁺24], who present an ISE for RISC-V that supports for multiply-accumulate. We argue that the present paper generalises the design in [CFG⁺24], which, for example, supports 1) RV64I and so $w = 64$ only, and 2) full-radix or one specific reduced-radix case where b is fixed to 57, motivated by CSIDH-512 [CLM⁺18] as the only use-case.

3 Design

To explore their relative advantages and disadvantages, we consider a set of 4 specific ISE variants which stem from 2 properties. As a means of explaining those properties, consider the generic description of a multiply-accumulate operation $\text{GPR}[\text{rd}] \leftarrow \text{MACCLU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}], c)$ which computes $\text{LSB}_w(\text{GPR}[\text{rs1}] \times \text{GPR}[\text{rs2}] + \text{GPR}[\text{rs3}])$ using the control value c to, e.g., select the radix used.

Property 1. A non-stateful (or *stateless*) ISE variant captures c using an immediate operand encoded in the instruction; in contrast, a stateful ISE variant captures c using dedicated Control and Status Register (CSR) accessed by the instruction.

We use the first available⁴ unprivileged, custom read/write CSR where necessary, referring to it as $\text{CSR}[801_{(16)}] \equiv \text{CSR}[\text{macc}]$. We assume a 4-bit $c \in \{0, 1, \dots, 15\}$, interpreted to

⁴The choice here is somewhat arbitrary, but stems from 1) use of the custom read/write address range, which, per [RIS24b, Table 3], is $800_{(16)}$ to $8FF_{(16)}$, then 2) selection of the first available CSR in said range, which, for the CVA6 base core is $801_{(16)}$ (because $800_{(16)}$ is used).

Destructive		Stateful		313029282726252423222120191817161514131211109876543210																	
Class-1			$\left\{ \begin{array}{l} \text{srl} \\ \text{iadd rd, rs1, rs2, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{SRLIADD}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
			$\left\{ \begin{array}{l} \text{srai} \\ \text{iadd rd, rs1, rs2, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{SRAIADD}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
	false		$\left\{ \begin{array}{l} \text{cacc rd, rs1, rs2, rs3} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{CACC}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}])$	313029282726252423222120191817161514131211109876543210																
	true		$\left\{ \begin{array}{l} \text{cacc rd, rs1, rs2} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{CACC}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rd}])$	313029282726252423222120191817161514131211109876543210																
Class-2			$\left\{ \begin{array}{l} \text{macclu rd, rs1, rs2, rs3, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCLU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
	false		$\left\{ \begin{array}{l} \text{macclu rd, rs1, rs2, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCLU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rd}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
	false		$\left\{ \begin{array}{l} \text{macclu rd, rs1, rs2, rs3} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCLU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}], \text{CSR}[\text{macc}])$	313029282726252423222120191817161514131211109876543210																
	true		$\left\{ \begin{array}{l} \text{macclu rd, rs1, rs2} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCLU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rd}], \text{CSR}[\text{macc}])$	313029282726252423222120191817161514131211109876543210																
	false		$\left\{ \begin{array}{l} \text{macchu rd, rs1, rs2, rs3, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCHU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
	true		$\left\{ \begin{array}{l} \text{macchu rd, rs1, rs2, imm} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCHU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rd}], \text{imm})$	313029282726252423222120191817161514131211109876543210																
	false		$\left\{ \begin{array}{l} \text{macchu rd, rs1, rs2, rs3} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCHU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rs3}], \text{CSR}[\text{macc}])$	313029282726252423222120191817161514131211109876543210																
	true		$\left\{ \begin{array}{l} \text{macchu rd, rs1, rs2} \end{array} \right\} \mapsto$	$\text{GPR}[\text{rd}] \leftarrow \text{MACCHU}(\text{GPR}[\text{rs1}], \text{GPR}[\text{rs2}], \text{GPR}[\text{rd}], \text{CSR}[\text{macc}])$	313029282726252423222120191817161514131211109876543210																
						313029282726252423222120191817161514131211109876543210															
						313029282726252423222120191817161514131211109876543210															
						313029282726252423222120191817161514131211109876543210															

Table 2: A summary of additional instructions that constitute the ISE, described in terms of assembly language syntax (left) plus encoding and semantics (right).

yield an implied radix of 2^b for $b = (w - 15) + c$. Put more simply, therefore,

$$\begin{aligned} c = 15 &\Rightarrow \text{full-radix} \Rightarrow \text{implied radix is } 2^b \text{ for } b = w \\ c < 15 &\Rightarrow \text{reduced-radix} \Rightarrow \text{implied radix is } 2^b \text{ for } w - 15 \leq b < w \end{aligned}$$

implies support for a useful subset of reduced-radix cases *close* to the full-radix case.

Property 2. A non-destructive ISE variant allows a free choice of **rd**, **rs1**, **rs2**, and **rs3**; in contrast, a destructive ISE variant implicitly defines **rs3** = **rd** and thus forces “destruction” (or overwriting) of that source operand.

Each ISE variant includes exactly 5 instructions, namely **srliadd**, **sraiadd**, **cacc**, **macclu**, and **macchu**. At a low-level, Table 2 summarises the encoding and semantics of said instructions; these details depend on the variant, and hence their (non-)stateful’ness and (non-)destructive’ness properties. At a high-level, however, the rationale for their design is to accelerate the performance-critical steps (e.g., those within the inner-most loop) in algorithms such as but not limited to Algorithm 1 and Algorithm 2. Dividing the instructions into two classes, we expand on this rationale in the following Sections.

3.1 Instruction class 1: support for carry propagation

This instruction class contains 3 instructions whose purpose is to support efficient carry propagation operations. The instruction semantics captured by Table 2 make use of the following algorithms:

```
1 Algorithm 3. SRLIADD( $x, y, imm$ ) begin
2 | return  $x + \text{ext}_w^0(y \gg imm)$ 
3 end
```

```
1 Algorithm 4. SRAIADD( $x, y, imm$ ) begin
2 | return  $x + \text{ext}_w^\pm(y \gg imm)$ 
3 end
```

```
1 Algorithm 5. CACC( $x, y, z$ ) begin
2 | return  $((x + y) \gg w) + z$ 
3 end
```

Noting that concrete examples are offered in Section 4.2, instructions in this class can be intuitively understood as supporting the combination (i.e., addition) of the Most-Significant Bits (MSBs) of one value with another. For example, based on the word size w , **CACC**(x, y, z) (i.e., Algorithm 5) will 1) compute the sum $t = x + y$, 2) extract the MSBs of t , then 3) add those MSBs to z .

3.2 Instruction class 2: support for multiply-accumulate

This instruction class contains 2 instructions whose purpose is to support efficient (unsigned, integer) multiply-accumulate operations. The instruction semantics captured by Table 2 make use of the following algorithms (left) and control functions (right, which are essentially constant values selected by w and c):

<pre>1 Algorithm 6. MACCLU(x, y, z, c) begin 2 $t_0 \leftarrow x \times y$ 3 if $f_0(c) = \text{true}$ then $t_1 \leftarrow z$ else $t_1 \leftarrow 0$ 4 if $f_0(c) = \text{true}$ then $t_2 \leftarrow 0$ else $t_2 \leftarrow z$ 5 $m \leftarrow (1 \ll f_1(c)) - 1$ 6 $r \leftarrow (((t_0 + t_1) \gg f_2(c)) \wedge m) + t_2$ 7 return r 8 end</pre>	$f_0(c) = \begin{cases} 1 & \text{if } c = 15 \\ 0 & \text{if } c \neq 15 \end{cases}$ $f_1(c) = \begin{cases} w & \text{if } c = 15 \\ (w - 15) + c & \text{if } c \neq 15 \end{cases}$ $f_2(c) = \begin{cases} 0 & \text{if } c = 15 \\ 0 & \text{if } c \neq 15 \end{cases}$
---	---

<pre> 1 Algorithm 7. $\text{MACCHU}(x, y, z, c)$ begin 2 $t_0 \leftarrow x \times y$ 3 if $f_3(c) = \text{true}$ then $t_1 \leftarrow z$ else $t_1 \leftarrow 0$ 4 if $f_3(c) = \text{true}$ then $t_2 \leftarrow 0$ else $t_2 \leftarrow z$ 5 $m \leftarrow (1 \ll f_4(c)) - 1$ 6 $r \leftarrow (((t_0 + t_1) \gg f_5(c)) \wedge m) + t_2$ 7 return r 8 end </pre>	$f_3(c) = \begin{cases} 1 & \text{if } c = 15 \\ 0 & \text{if } c \neq 15 \end{cases}$ $f_4(c) = \begin{cases} w & \text{if } c = 15 \\ w & \text{if } c \neq 15 \end{cases}$ $f_5(c) = \begin{cases} w & \text{if } c = 15 \\ (w - 15) + c & \text{if } c \neq 15 \end{cases}$
--	---

Noting that concrete examples are offered in Section 4.2, instructions in this class can be intuitively understood as extensions of `mul` and `mulhu` from the base ISA to include an accumulation step. For example, based on the word size w and implied radix $2^b \leq 2^w$, $\text{MACCLU}(x, y, z, c)$ (i.e., Algorithm 8) will 1) compute the product $t = x \times y$, 2) form a set of masks, then 3) shift and mask t , then add the result to z .

3.3 Discussion

Trade-offs around radix selection and switching. Selecting between non-stateful or stateful ISE variants can be viewed as a trade-off between efficiency versus flexibility of radix selection based the following points. First, a non-stateful ISE variant implies lower-latency radix switching because the radix can be selected and so switched on a per-instruction basis; a stateful ISE variant implies higher-latency⁵ radix switching, in contrast, because an update of `CSR[mac]` is required to switch the radix. Second, a non-stateful ISE variant disallows dynamic radix switching because the radix must be selected and encoded statically in a given instruction; a stateful ISE variant allows dynamic radix switching, in contrast, because the radix can be switched by updating `CSR[mac]`. This fact can have an impact on the wider implementation strategy used. For example, use of a non-stateful ISE variant alongside loop unrolling demands co-selection of the radix and unrolling factor because the unrolled instruction instances need to include the radix in their encoding; this is not true of a stateful ISE variant, where the radix and unrolling factor can be controlled independently.

Note that use-cases certainly exist where radix switching is a concern: examples include the so-called “ref 10” implementation of Curve25519 due to Bernstein, which uses a mixture of 25- and 26-bit limbs and so suggests a requirement for frequent radix switching. However, the more common use-case would see one radix selected then used for a significant amount of computation; the latter will almost certainly amortise any overhead of the former.

Trade-offs around instruction encoding. We fix our scope to use of standard *fixed*-length, 32-bit instructions; effective use of *variable*-length instructions (e.g., sequences of 16-bit instruction “parcels” [RIS24a, Section 1.5]) is left as an open question for future work. A RISC-V integer ALU-like instruction such as `add` is encoded using the 3-address R-type instruction format [RIS24a, Section 2.2], which involves 2 source and 1 destination register addresses. `macclu` and `macchu` deviate from this, however, necessarily adopting a 4-address instruction format which involves 3 source and 1 destination register addresses. Note that said format is evident in both the non-destructive and destructive ISE variants, even though the addresses are encoded differently: in the former `rs3` is an explicit field, whereas in the latter `rs3` is an implicit field equal to `rd`. Also note that RISC-V already includes the 4-address R4-type instruction format [RIS24a, Section 20.6] which is conceptually

⁵Update of `CSR[mac]` involves execution of at least 1 instruction, which one might hope has a 1-cycle latency. However, this ideal might not be reflected by a given micro-architecture. In the CVA6 base core, for example, such an instruction must fully retire before a value is written-back and is then visible to or influences subsequent instructions; doing so can take significantly more than 1 cycle.

similar. We avoid using this, however, due to it having a remit limited to floating-point ALU-like instructions such as `fmadd`.

To both explain the design and inform an associated implementation, the following points are important. First, if one ignores the potential for multi-cycle operand fetch, use of a format with 3 source operands implies an additional read port on the general-purpose register file. Second, `macclu` and `macchu` can be viewed as producing the less- and more-significant halves of the product: the rationale is that doing so mirrors `mul` and `mulhu` by retaining a format with 1 destination operand, and so avoids an additional write port on the general-purpose register file. We note that instances of `macclu` and `macchu` can be fused (see, e.g., [CDPA16, SPJR22]) as a means of mitigating any resulting overhead.

Support for a sparse set of implied radix. The design presented supports an implied radix of 2^b for $w - 15 \leq b \leq w$, with $b < w$ (resp. $b = w$) leading to a reduced-radix (resp. the full-radix) case. At least in theory, it might be plausible to allow a larger set of implied radix to be specified versus actually implemented in hardware. Doing so might allow the hardware to be specialised (e.g., for $b = 57$ only, per [CFG⁺24]), but implies a need for discoverability of supported versus unsupported cases: raising an exception for the latter is at least possible, but not very elegant.

Features of the multiply-accumulate data-path. The algorithmic description of `MACCLU` and `MACCHU` is intentionally structured in order to highlight similarity of the two cases. The difference between them is essentially the control functions, i.e., the set of f_i used, meaning that a single data-path can support both. To both explain the design and inform an associated implementation, the following points are important.

First, Line 2 is common to both: said data-path could therefore be viewed as post-processing the output of an *existing* ($w \times w$)-bit integer multiplier. When viewed as such, it should be clear that if the `mul` and `mulhu` instructions are already Zkt-compliant, it is plausible, even straightforward, to ensure that `macclu` and `macchu` are as well. Second, although Lines 3 and 4 each seem to imply a 2-input, w -bit multiplexer, the selection between 0 and z can instead be realised using w AND gates.

Support for full-radix MPI addition. In combination with `add`, the `cacc` instruction can be used to speed up not only full-radix MPI multiplication, but also MPI addition. When the target register of the `add` differs from its source registers, it is possible to execute the `add` and the associated `cacc` instruction in arbitrary order, thereby enabling instruction-level parallelism in a superscalar architecture. This contrasts with combinations of `add` and `sltu` for carry propagation, which always require a sequential execution (i.e., the `sltu` can only be executed *after* the `add`) that inhibits instruction-level parallelism.

4 Implementation

In this Section, we focus on two perspectives related to implementation of the ISE design presented in Section 3: in Section 4.1 we adopt hardware-oriented perspective by detailing how the ISE is realised in the CVA6 base core. whereas in Section 4.2 we adopt software-oriented perspective by detailing how the ISE is utilised.

4.1 Hardware-oriented implementation

Base core: CVA6. Having been initially created at ETHZ [ZB19] under the name Ariane, CVA6⁶ is a RISC-V compliant micro-processor core now (actively) developed under

⁶<https://github.com/openhwgroup/cva6>

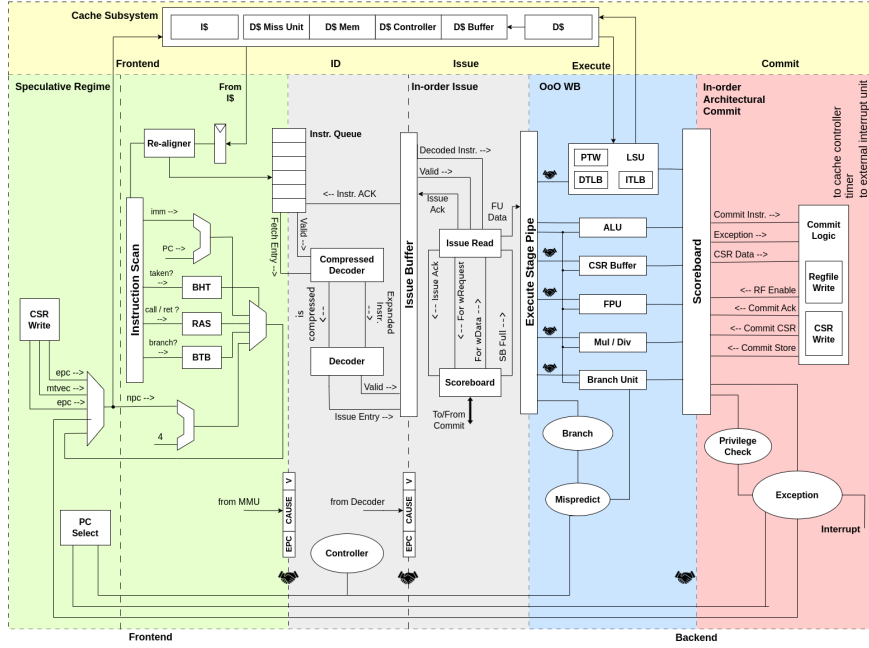


Figure 1: A higher-level block diagram describing the generic CVA6 micro-architecture (source: `ariane_overview.drawio.png`, taken from <https://github.com/openhwgroup/cva6>).

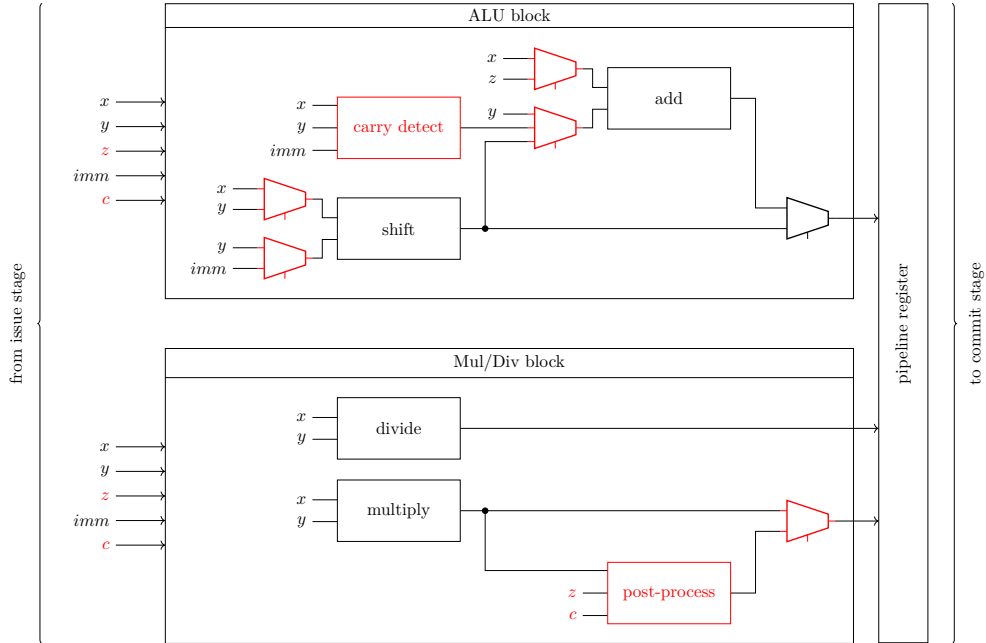


Figure 2: A lower-level block diagram describing the ISE implementation within the Arithmetic and Logic Unit (ALU) and integer multiplier (labelled Mul/Div in Figure 1) blocks which form part of the CVA6 execute pipeline stage. Components that exist within (resp. added by) the base core (resp. extended core) are shown in black (resp. red).

management of the OpenHW Group. More accurately, CVA6 supports multiple specific cores derived from the same SystemVerilog-based source code base. Such a core is specified by selecting a value for (more than 50) fine-grained parameters; a set of such parameters, termed a configuration, is then used to specialise Register-Transfer Level (RTL) which implements the 6-stage pipeline illustrated by Figure 1. The parameters can be roughly classified as relating to 1) the ISA (e.g., 32-bit RV32I versus 64-bit RV64I, plus common extensions such as M, A, F, and C), 2) the micro-architecture (e.g., cache and Translation Look-aside Buffer (TLB) organisation, scalar versus super-scalar instruction execution), and 3) the implementation technology (e.g., Field Programmable Gate Array (FPGA) versus Application Specific Integrated Circuit (ASIC)). This high degree of configurability allows CVA6-based cores to be viable in, e.g., application-class and embedded-class systems.

Although one can develop configurations from scratch, a number (7 based on RV32I, 13 based on RV64I) of pre-developed, tested and somewhat diverse instances exist. We opted to base our implementation on the 32-bit `cv32a6_ima6fc_sv32`⁷ and 64-bit `cv64a6_ima6fdc_sv39`⁸ configurations; doing so was influenced by several factors, including functional correctness guarantees, micro-architectural similarity between 32- and 64-bit cases, and support for simulated and FPGA-based development and evaluation. Although imprecise, we use the short-hand terms CV32A6 and CV64A6 to refer to the respective base cores from here on.

Extended core: CVA6 + ISE. To prototype an implementation of the ISE variants outlined in Section 3, we take a base core and alter it to yield an extended core; due to uniformity of the ISE design, the alterations are essentially the same for either CV32A6 or CV64A6. Specifically, we adopt a structured approach that involves 4 steps which we label as either (G)eneric (if the step is applied in the same way to all ISE variants) or (S)pecific (if the step is applied on a per-variant basis):

1. (G) We altered the general-purpose register file to support the 4-address instruction format required. As well as adding an additional read port to the register file itself, doing so implies extension of the associated dependency checking and operand forwarding logic: we added an operand bus from the register file to the ALU and integer multiplier blocks, for example.
2. (S) For stateful ISE variants, we altered the CSR register file by adding `CSR[macc]`.
3. (S) We altered the decode stage to identify ISE-based instructions, and derive associated control signals. A central example of such a control signal is `c`, which, per Section 3, specifies the implied radix. For stateful ISE variants `c` is read from `CSR[macc]`, whereas for non-stateful ISE variants `c` is decoded from appropriate (immediate) fields in the instruction; in either case, `c` then forms part of the instruction packet eventually issued to and thus used by the execute stage.
4. (G) We altered the execute stage to support computation related to ISE-based instructions. Rather than dedicated functional units or a CVA6-supported extension mechanism such as CV-X-IF, we instead implemented tightly-integrated support within the existing ALU or integer multiplier blocks; the former integrates combinatorial logic that supports `SRLIADD`, `SRAIADD`, and `CACC`. whereas the latter integrates combinatorial logic that supports `MACCLU` and `MACCLU`. As highlighted by Figure 2, this approach allows reuse of existing components within said blocks, motivated, e.g., by minimisation of overhead. A significant example is reuse the existing integer multiplier to support `MACCLU` and `MACCLU`, implemented by post-processing the output per Section 3.

⁷https://github.com/openhwgroup/cva6/blob/master/core/include/cv32a6_ima6fc_sv32_config_pkg.sv

⁸https://github.com/openhwgroup/cva6/blob/master/core/include/cv64a6_ima6fdcv_sv39_config_pkg.sv

Listing 1: RISC-V assembly implementation of multiply-add operation of the form $s \leftarrow a \cdot b + t + z$, which is executed in the inner loop of the OSM.

```
.macro MADD_V3 shi, slo, ain, \
                bin, tin, zin
    mul    tmp0, \ain, \bin
    mulhu  tmp1, \ain, \bin
    add    \slo, tmp0, \zin
    sltu   tmp0, \slo, tmp0
    add    tmp1, tmp1, tmp0
    add    \slo, \slo, \tin
    sltu   tmp0, \slo, \tin
    add    \shi, tmp1, tmp0
.endm
```

Listing 2: RISC-V assembly implementation of multiply-accumulate operation of the form $s \leftarrow s + a \cdot b$, which is executed in the inner loop of the PSM.

```
.macro MACC_V4 sxt, shi, slo, \
                ain, bin
    mul    tmp0, \ain, \bin
    mulhu  tmp1, \ain, \bin
    add    \slo, \slo, tmp0
    sltu   tmp0, \slo, tmp0
    add    tmp1, tmp1, tmp0
    add    \shi, \shi, tmp1
    sltu   tmp1, \shi, tmp1
    add    \sxt, \sxt, tmp1
.endm
```

Using the base core configuration, we create further sub-configurations to capture (non-)statefulness and (non-)destructiveness parameters and thus allow specification of each ISE variant, e.g., during synthesis or simulation of the resulting extended core.

4.2 Software-oriented implementation

In this subsection, we focus on the most performance-critical operations of MPI multiplication, which are the operations performed in the inner loop of the OSM and PSM, respectively. As explained in Section 2.3, the OSM executes a multiply-add operation of the form $s \leftarrow a \cdot b + t + z$ in its inner loop, whereas the inner-loop operation of the PSM is a classical multiply-accumulate of the form $s \leftarrow s + a \cdot b$. Listing 1 and Listing 2 show optimized assembly code fragments for the two inner-loop operations, whereby both implementations assume a full-radix representation of the operands. Somewhat surprisingly, the two inner-loop operations do not only have the same instruction-count, namely eight, but they also execute the same type of instruction: one `mul`, one `mulhu`, four `add` and two `sltu`. The multiply-add operation of the OSM stores the double-word sum s in the two registers `shi` and `slo`, while the multiply-accumulate operation of the PSM requires a triple-word accumulator (registers `sxt`, `shi` and `slo`).

When taking advantage of our proposed ISE, the number of instructions in both cases drops from eight to only four; the corresponding ISE-supported assembly macros are shown in Listing 3 (OSM) and Listing 4 (PSM). The two custom instructions `macclu` and `macchu` compute the lower and upper word of the product of two words a and b (in registers `ain` and `bin`), to which a third word (either `zin` or `slo`) is added. This operation is a part of both the multiply-add of the OSM and the multiply-accumulate of the PSM. The remaining part is the addition of another word (either `tin` or `shi`) to the result produced by `macclu` and `macchu`, along with a carry propagation, which is accelerated through the `cacc` instruction. Consequently, the instruction count of the inner-loop operation of both the OSM and the PSM is reduced by half, which means both basic multiplication methods benefit equally from our ISE.

As mentioned in Section 2.3, adopting a reduced-radix representation makes mainly sense for the PSM as it allows one to store the cumulative sum in two instead of three accumulator registers, thereby reducing the effort for accumulation (and associated carry-propagation) of the products of operand-limbs. As a consequence, a reduced-radix representation will reduce the number of instructions to be carried out in the inner loop, but, as explained in Section 2.3, this saving normally comes at the expense of an increased number of iterations. On the other hand, using a reduced-radix representation for the OSM is normally not beneficial, since the inner loop would still require the same number

Listing 3: ISE-supported assembly implementation of multiply-add operation of the form $s \leftarrow a \cdot b + t + z$, which is executed in the inner loop of the OSM.

```
.macro MADD_V3 shi, slo, ain, \
                bin, tin, zin
    macclu tmp0, \ain, \bin, \zin
    macchu tmp1, \ain, \bin, \zin
    add    \slo, \tin, tmp0
    cacc   \shi, \tin, tmp0, tmp1
.endm
```

Listing 4: ISE-supported assembly implementation of multiply-accumulate operation of the form $s \leftarrow s + a \cdot b$, which is executed in the inner loop of the PSM.

```
.macro MACC_V4 sxt, shi, slo, \
                ain, bin
    macchu tmp0, \ain, \bin, \slo
    macclu \slo, \ain, \bin, \slo
    cacc   \sxt, \shi, tmp0, \sxt
    add    \shi, \shi, tmp0
.endm
```

of instructions as in the full-radix case.

Listing 5: RISC-V assembly implementation of the PSM multiply-accumulate operation in the reduced-radix case.

```
.macro MACC_V4 shi, slo, ain, bin
    mul    tmp1, \ain, \bin
    mulhu  tmp0, \ain, \bin
    add    \slo, \slo, tmp1
    sltu   tmp1, \slo, tmp1
    add    tmp0, tmp0, tmp1
    add    \shi, \shi, tmp0
.endm
```

Listing 6: ISE-supported assembly implementation of the PSM multiply-accumulate operation in the reduced-radix case.

```
.macro MACC_V4 shi, slo, ain, bin
    macclu \slo, \ain, \bin, \slo
    macchu \shi, \ain, \bin, \shi
.endm
```

Listing 5 shows the assembly code of the PSM multiply-accumulate operation when using a reduced-radix representation where the limb-products are accumulated into a two-register accumulator (`shi` and `slo`). As expected, the reduced-radix representation saves two instructions compared to the full-radix case, i.e., the instruction-count is reduced from eight to six. The corresponding ISE-supported implementation of the macro, consisting of only the two custom instructions `macclu` and `macchu`, is contained in Listing 6. Our X25519 implementation adopts a reduced-radix implementation with a fixed limb-length of $b = 29$ for CV32A6 and $b = 51$ for CV64A6. The limb-length of the case study on MPI multiplication is selected depending on the operand-length so as to not overflow the two accumulator registers.

5 Evaluation

In this Section, we focus on two perspectives related to evaluation of the ISE design presented in Section 3: in Section 5.1 we adopt hardware-oriented perspective by focusing on, e.g., area and timing overhead, whereas in Section 5.2 we adopt software-oriented perspective by focusing on, e.g., execution latency. To support the content, we consider 2 different synthesis targets and so associated tool-chains.

FPGA-based synthesis target. In this case, we consider the CVA6-based core instantiated within a so-called Core-V Accelerated Processing Unit (APU), i.e., a complete System-on-Chip (SoC): this platform includes a comprehensive set of peripherals interconnected via an Advanced eXtensible Interface (AXI) bus, and thereby supports execution of a Linux-based Operating System (OS) kernel built from-source using the CVA6 Software

Table 3: A comparison of area, i.e., hardware overhead, covering ISA-only and ISE-supported cases for both CV32A6 and CV64A6: the results stem from synthesis of the base core versus the base core plus a given variant of the ISE, for both FPGA- and ASIC-focused synthesis targets.

Platform	Core	Destr.	Stateful	FPGA			ASIC
				LUTs	Regs	Other	
CV32A6	Base core			23,871 (0%)	11,245 (0%)		85,584 (0%)
	Base core + ISE	false	false	24,673 (3.4%)	11,432 (1.7%)	41 BRAMs	88,672 (3.6%)
		true	false	24,679 (3.4%)	11,432 (1.7%)	+	89,303 (4.4%)
		false	true	24,880 (4.2%)	11,441 (1.7%)	6 DSPs	88,867 (3.8%)
		true	true	24,881 (4.2%)	11,441 (1.7%)		89,161 (4.2%)
CV64A6	Base core			45,239 (0%)	23,457 (0%)		203,742 (0%)
	Base core + ISE	false	false	47,139 (4.2%)	23,921 (2.2%)	36 BRAMs	212,031 (4.1%)
		true	false	47,091 (4.1%)	23,922 (2.2%)	+	211,841 (4.0%)
		false	true	47,267 (4.5%)	23,922 (2.2%)	27 DSPs	211,794 (4.0%)
		true	true	47,237 (4.4%)	23,921 (2.2%)		212,573 (3.8%)

Development Kit (SDK)⁹. We synthesise the associated RTL using Xilinx Vivado version 2024.1; the target FPGA is a Xilinx Kintex-7 (model `xc7k325t-2ffg900c`), mounted on a Genesys 2¹⁰ development board. The FPGA uses a 200 MHz external clock input, which is routed to a clock generator module: this produces 1) the primary 50 MHz clock signal for use by the core, 2) two secondary 125 MHz clock signals for use by the high-speed peripherals. Default synthesis settings are used, with no effort invested into identifying nor capitalising on additional opportunities for optimisation.

ASIC-based synthesis target. In this case, we consider the CVA6-based core as a standalone component. We synthesise the associated RTL using from-source¹¹ builds of `yosys` (commit `cd71f190c`)¹² and `yosys-slang` (commit `6ba28d9`)¹³. The process involves four main steps: 1) the SystemVerilog RTL is provided as input to, and then parsed by the `yosys-slang` frontend, 2) a flattened, technology-independent netlist is produced using the `yosys synth -flatten -noabc` command, 3) this netlist is mapped to the NanGate45 standard cell library¹⁴ and undergoes logic optimization using ABC¹⁵, then 4) post-synthesis statistics are generated: the `stat` command reports area based on cell count, while the `ltp` (longest topological path) command provides a technology-independent estimate of critical path.

5.1 Evaluating hardware-oriented metrics: ISE implementation

In this Section, we focus on implementation of the ISE, i.e., adopt a hardware-focused perspective. For both the CV32A6 and CV64A6 configurations, Table 3 summarises results for two cases: these are 1) the base core, and 2) the base core plus a given ISE variant, with the results split into a left-hand side relating to the FPGA-based synthesis target, and a right-hand side relating to the ASIC-based synthesis target.

⁹<https://github.com/openhwgroup/cva6-sdk>

¹⁰<https://digilent.com/reference/programmable-logic/genesys-2/reference-manual>.

¹¹Both `yosys` and `yosys-slang` are in active development; we avoided use of stable, release versions because they did not (yet) include crucial bug fixes that allowed correct synthesis of CVA6.

¹²<https://github.com/YosysHQ/yosys>

¹³<https://github.com/povik/yosys-slang>

¹⁴https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/blob/master/flow/platform/s/nangate45/lib/NangateOpenCellLibrary_typical.lib.

¹⁵<https://people.eecs.berkeley.edu/~alanmi/abc>.

5.1.1 Area

For the FPGA-based synthesis target, a result is shown for each relevant resource; from a positive perspective this offers more comprehensive, fine-grained detail, but from a negative perspective it makes interpretation of the overhead somewhat harder. For example, depending on the ISE variant and focused¹⁶ on Look-Up Table (LUT) usage alone, the overhead is between 3.4% and 4.2% for CV32A6, and between 4.1% and 4.5% for CV64A6. For the ASIC-based synthesis target, a single, consolidated result is shown, i.e., total number of NAND2 cells. For example, depending on the ISE variant, the overhead is between 3.6% and 4.4% for CV32A6, and between 3.8% and 4.1% for CV64A6.

In summary, we conclude that the area overhead is 1) modest, but still significant relative to the base core, and 2) largely independent of the ISE variant, with none clearly preferable based on this metric alone. Beyond this, however, one might consider ISE-agnostic (i.e., related to overhead that might be amortised by supporting use beyond the ISE) from ISE-specific sources of overhead. One instance of the former is micro-architectural support for a 4-address instruction format; clearly this represents overhead associated with the ISE, but, equally, *could* have more general utility for *any* instruction¹⁷ implying a need for an additional read port(s) on the general-purpose register file.

5.1.2 Timing

For the ASIC-based synthesis target, no timing analysis was performed: the remit of `yosys` is synthesis only. For the FPGA-based synthesis target, however, we used the timing report generated by Vivado to investigate whether and what impact implementation of an ISE variant had. Said report details the worst negative slack for the 100 longest critical paths identified at the SoC level. In a case where the integer multiplier determines the critical path, treating the data-path supporting MACCLU and MACCHI as post-processing the integer multiplier output could clearly extend this. However, we found only 1 of the 100 critical paths reported was within the core itself; this related to the debug module, with non-core examples relating, e.g., to DRAM, AXI, and Ethernet interfaces, etc. At least for the base core and configurations of it used, we concluded that implementation of no ISE variant 1) impacted the CVA6 critical path nor, therefore, 2) the maximum possible clock frequency.

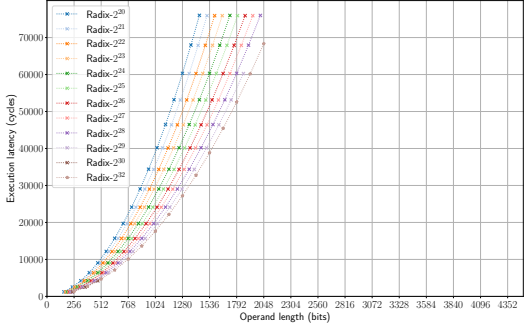
5.2 Evaluating software-oriented metrics: ISE utilisation

In this Section, we focus on utilisation of the ISE, i.e., adopt a software-focused perspective centered around execution latency. To do so we capture the kernel of interest in a benchmark harness which, at a high level, adopts a 2-phase strategy. First, in a pre-measurement phase, we perform steps such as cache warming. Second, in a measurement phase, we use the RISC-V CSR[cycle] cycle counter CSR (see, e.g., [RIS24a, Section 7]) to measure mean (and variance of) execution latency over a large number of trials. We compile the benchmark using version 13.1.0 of the GCC-based RISC-V tool-chain¹⁸ with optimisation implied by the `-O3` flag, then use a Verilator-based, simulated but cycle accurate platform (i.e., the synthesised CVA6-based core plus memory hierarchy) to execute it; doing so avoids the need for a post-measurement phase to address systemic noise and outlier measurements

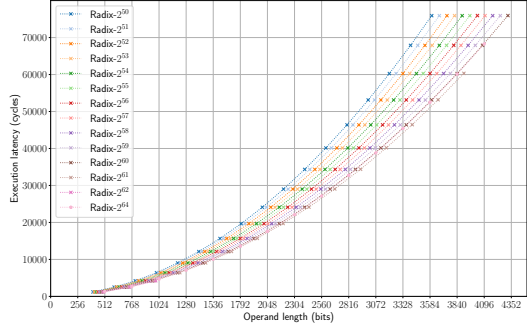
¹⁶Note that 1) use of Digital Signal Processing (DSP) resources is localised to the Floating-Point Unit (FPU) implementation, and 2) use of Block RAM (BRAM) resources is localised to the memory subsystem, specifically the caches and TLB; although there is some differences between CV32A6 and CV64A6 (e.g., due to differences in their cache parameterisation), neither is impacted by the ISE implementation.

¹⁷For example, within the context of RV32I, the Zdinx [RIS24a, Section 26] extension supports computation with double-precision 64-bit floating-point values stored in a register pair [RIS24a, Section 26.3] (i.e., two 32-bit general-purpose registers); 2 such pairs would require 4 reads from the general-purpose register file, which, to deliver single-cycle execution, would imply a need for 4 read ports.

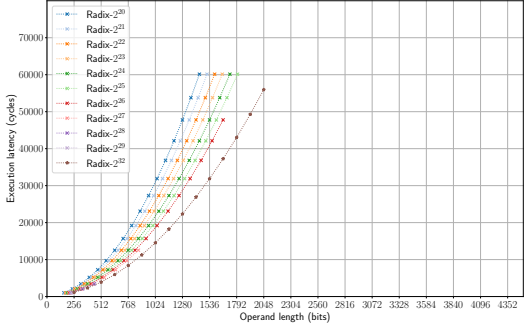
¹⁸<https://github.com/riscv-collab/riscv-gnu-toolchain>



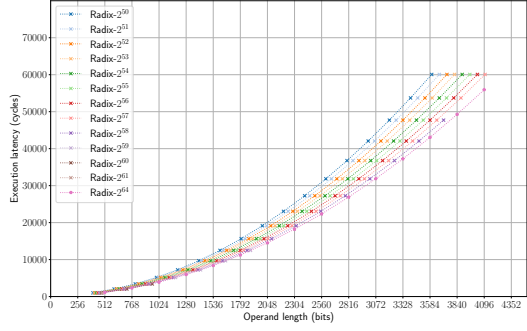
(a) CV32A6, ISA-only, simple.



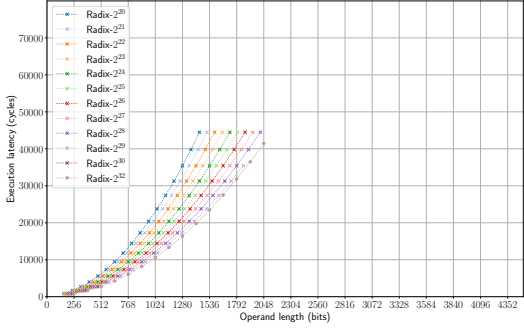
(b) CV64A6, ISA-only, simple.



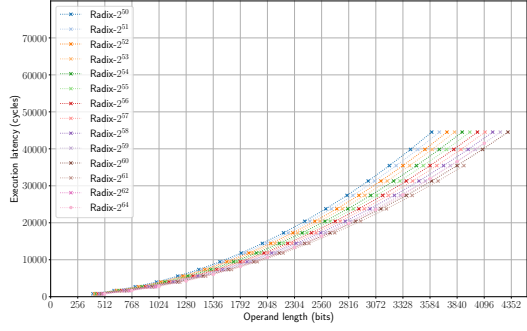
(c) CV32A6, ISE-supported, simple.



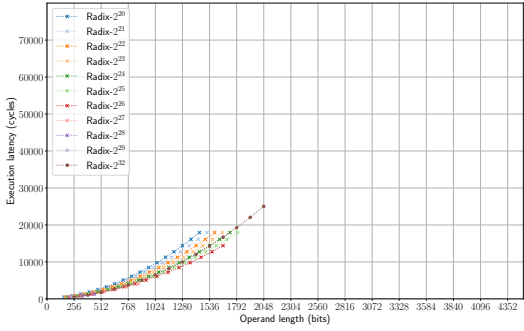
(d) CV64A6, ISE-supported, simple.



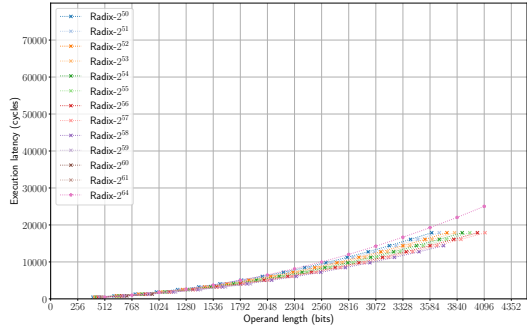
(e) CV32A6, ISA-only, hybrid.



(f) CV64A6, ISA-only, hybrid.



(g) CV32A6, ISE-supported, hybrid.



(h) CV64A6, ISE-supported, hybrid.

Figure 3: A comparison of execution latency (measured in cycles) for n -bit integer multiplication, covering ISA-only and (non-stateful, non-destructive) ISE-supported cases for CV32A6 and CV64A6: the results capture data points for full-radix (i.e., radix-2 ^{b} where $b = w$) and reduced-radix (i.e., radix-2 ^{b} where $b < w$) cases.

Table 4: A comparison of execution latency (measured in cycles) for X25519-related arithmetic, covering ISA-only and (non-stateful, non-destructive) ISE-supported cases for CV32A6 and CV64A6: the results capture data points for a range of n , plus both full-radix (i.e., radix- 2^b where $b = w$) and reduced-radix (i.e., radix- 2^b where $b = 51$ for CV64A6, or $b = 29$ for CV32A6) cases. Note that multiply-full and multiply-word refer to computation of a product $r = x \times y \pmod{p}$ where y is a full field element and upto w bits respectively: the latter caters for multiplication by small constants.

Platform	Operation	Full-radix		Reduced-radix	
		ISA-only	ISA + ISE	ISA-only	ISA + ISE
CV32A6	$\mathbb{F}_p$ add	98	93	89	89
	$\mathbb{F}_p$ subtract	111	107	94	94
	$\mathbb{F}_p$ square	527	350	554	374
	$\mathbb{F}_p$ multiply-full	689	410	789	411
	$\mathbb{F}_p$ multiply-word	114	83	144	107
	$\mathbb{F}_p$ invert	142,055	92,802	150,035	97,858
	$E(\mathbb{F}_p)$ ladder step	6,305	4,121	6,820	4,189
	$E(\mathbb{F}_p)$ scalar mult.	1,857,049	1,222,651	2,010,881	1,256,837
CV64A6	$\mathbb{F}_p$ add	48	45	40	40
	$\mathbb{F}_p$ subtract	60	57	44	44
	$\mathbb{F}_p$ square	160	108	167	84
	$\mathbb{F}_p$ multiply-full	190	115	245	125
	$\mathbb{F}_p$ multiply-word	48	38	83	49
	$\mathbb{F}_p$ invert	43,170	28,830	45,283	22,647
	$E(\mathbb{F}_p)$ ladder step	2,015	1,397	2,253	1,291
	$E(\mathbb{F}_p)$ scalar mult.	591,253	414,917	660,258	383,848

stemming, e.g., from interrupts and context switches. That said, we then also use an FPGA-based platform to validate the results by executing the same benchmark as a user-mode process, under control of Linux.

Note that, in the interests of brevity, Section 5.2.1, through Section 5.2.3 consider *only* what we view as the most attractive ISE variant in terms of functionality, namely the non-stateful, non-destructive case; we then dedicate Section 5.2.4 to discussion of the other cases.

5.2.1 Latency: per-instruction

The per-instruction latency of ISE instructions match the *structural latency* of counterparts in the base ISA, because their execution is supported by alteration to the existing ALU and integer multiplier blocks within the execute stage (i.e., the pipeline structure, and associated execution model remains unaltered). More specifically, Class 1 instructions (`sraiadd`, `sriiadd`, and `cacc`) share the same execution latency as instructions such as `add`; Class 2 instructions (`macchu` and `macclu`) share the same execution latency as instructions such as `mul`. However, a key distinction for Class 2 instructions is their 4-address format. While this does not alter the structural latency, it increases the potential for data-dependency stalls, which can lead to a higher *effective latency* compared to the standard 3-address format. Identifying and resolving such dependencies in the 4-address case can be approached in the same way as for the 3-address case, however: in our experience minimising or even eliminating stalls was possible, meaning this issue had negligible impact.

5.2.2 Latency: multi-precision integer multiplication

In this Section, we consider the execution latency of multi-precision integer multiplication, i.e., computation of the product $r = x \times y$ for n -bit x and y . For both the CV32A6 and CV64A6 configurations, Figure 3 summarises results for two cases: these are 1) use of the base ISA only versus ISA plus ISE, and 2) use of straightforward PSM versus hybrid implementation strategy, for a range of operand lengths, i.e., selections of n , and radix, i.e., selections of b leading to a radix- 2^b representation.

The simple implementation strategy reflects [MOV96, Algorithm 14.12] more or less as is: this is an operand-scanning variant of multiplication, processing 1 word/limb of the operands within the inner loop. The hybrid implementation strategy attempts to make best use of the (large, 32-entry) RISC-V general-purpose register file: it applies a limited form of operand caching, processing 4 limbs of the operands within the inner loop (see Sect. 2.3). Rather than select n , we instead select the number of limbs l and set $n = l \cdot 2^b$. However, for the hybrid implementation strategy this is a constrained selection: if $b < w$, i.e., a reduced-radix is used, use of the ISA demands $l \leq 2^{2(w-b)}$ whereas use of the ISE demands $l \leq 2^{w-b}$. Either way, the same sequence of instructions will therefore be executed for given an l (because it determines the number of loop iterations, for example).

This fact explains two features evident in Figure 3. First, the results for CV64A6 extend further towards the right-hand side than for CV32A6: given an l , b and hence $n = l \cdot 2^b$ is larger for CV64A6 than for CV32A6. Second, the results stemming from selection of b “spread” over a range of n for the same cycle count: given an l , $n = l \cdot 2^b$ is larger for a larger b . Beyond these, several general trends, i.e., across selections of n and b , are also evident. First, given some fixed operand length, various expected conclusions hold: for example, CV64A6 will generally result in lower latency than CV32A6, use of the ISE will generally result in lower latency than the ISA only, and use of the hybrid implementation strategy will generally result in lower latency than the simple implementation strategy. Second, it is difficult to draw a general conclusion about selection between full- versus reduced-radix: as one might expect, any cross-over point depends on various features of the implementation combined with the operand length. However, such cross-over points *do* exist. For both CV32A6 (Figure 3g) and CV64A6 (Figure 3h), for example, use of the ISE and a hybrid strategy leads to a reduced-radix being the most efficient option.

5.2.3 Latency: X25519

In this Section, we consider execution latency of the low- (e.g., field) and high-level (e.g., curve) arithmetic components underpinning X25519 [Ber06]. Versus the previous Section, this focus allows a more specific, in-context comparison between optimally parameterised (e.g., selection of the radix based on p) ISA- and ISE-based cases. For both the CV32A6 and CV64A6 configurations, Table 4 summarises results for two cases: these are 1) use of the base ISA only versus ISA plus ISE, and 2) use of full- versus reduced-radix. Again, various expected conclusions hold: for example, CV64A6 will generally result in lower latency than CV32A6, and use of the ISE will generally result in lower latency than the ISA only. Again, selection between full- versus reduced-radix depends on various features of the implementation. For CV32A6, full-radix is the most efficient selection whether using the ISA only (more significantly so) or the ISE (less significantly so); for CV64A6, reduced-radix is the most efficient selection whether using the ISA only or the ISE.

A headline conclusion from Table 4, and arguably the evaluation overall, is the improvement afforded by use of the ISE. Focusing on the top-level operation used by X25519, i.e., scalar multiplication on $E(\mathbb{F}_p)$, use of the ISE affords $1.5\times$ and $1.6\times$ improvement for full- and reduced-radix cases respectively on CV32A6; it affords $1.3\times$ and $1.7\times$ improvement for full- and reduced-radix cases respectively on CV64A6. For comparison, Scott’s C implementations of X25519 for 32-bit and 64-bit targets described in [Sco24] need 2,422,254

cycles on CV32A6 and 852,752 cycles on CV64A6, i.e., they are significantly slower than our ISA-only variants.

5.2.4 Latency: other ISE variants

All per-instruction execution latencies remains the same, irrespective of the ISE variant; the multi-instruction execution latencies, i.e., those quoted in Section 5.2.2 and Section 5.2.3 *are* influenced per the ISE properties. First, selection of a stateful (versus non-stateful) ISE variant implies negligible overhead re. execution latency: the reason is that the radix is selected once (i.e., there is no radix switching) meaning the overhead stems from essentially 1 instruction required to set CSR[macc]. Second, selection of a destructive (versus non-destructive) ISE variant implies roughly 9% overhead re. execution latency: the reason is that additional (typically mv) instructions are required to avoid destruction (i.e., overwriting) of live data which is subsequently used.

6 Conclusion

The software implementation of many cryptographic constructions depends on multi-precision integer arithmetic, the efficiency of which is impacted by the ISA such an implementation is expressed using. In this paper we attempt to bridge the gap between minimal support offered by RISC-V for unsigned integer multiplication, and richer support offered by ISAs such as ARMv7-M; specifically, we extend the former with a set of instructions related to unsigned integer multiply-accumulate.

Both ISA and so ISE design naturally involves consideration of trade-offs between points in a (potentially large) design space. Although different constraints might lead to different decisions, we argue that evaluation of our design demonstrates the overhead (e.g., in terms of area) required to realise the ISE is worthwhile given the degree of improvement in efficiency afforded by utilisation of it. Either way, however, the advantage it offers in terms of flexibility seems more clear cut: the (novel) feature of flexible radix selection enables important optimisation techniques at both algorithm and implementation levels, and so efficient support for the arithmetic requirements of the current- *and*, crucially, next-generation of cryptographic constructions.

Acknowledgements

This work has been supported in part by Innovate UK via project 10065634 (SCHEME: Safety Critical Harsh Environment Micro-processing Evolution), and in part by the National Natural Science Foundation of China via grant 62502275. The authors wish to thank the maintainers of both the CVA6 and `yosys-slang` projects for providing invaluable and timely support, which greatly facilitated the development and validation of the work described.

References

- [AP14] K. Asanović and D.A. Patterson. Instruction sets should be free: The case for RISC-V. Technical Report UCB/EECS-2014-146, 2014. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html>.
- [ARM18] ARMv6-M Architecture Reference Manual. Technical Report DDI0419E, ARM Ltd., 2018.

- [ARM21a] ARMv7-M Architecture Reference Manual. Technical Report DDI-0403E.e, ARM Ltd., 2021.
- [ARM21b] Armv8-M Architecture Reference Manual. Technical Report DDI-0553B.o, Arm Ltd., 2021.
- [Ber06] D.J. Bernstein. Curve25519: New Diffie-Hellman speed records. In *Public Key Cryptography (PKC)*, LNCS 3958, pages 207–228. Springer-Verlag, 2006. https://doi.org/10.1007/11745853_14.
- [CD23] W. Castryck and T. Decru. An efficient key recovery attack on SIDH. In *Advances in Cryptology (EUROCRYPT)*, LNCS 14008, pages 423–447. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-30589-4_15.
- [CDPA16] C. Celio, P. Dabbelt, D.A. Patterson, and K. Asanović. The renewed case for the reduced instruction set computer: Avoiding ISA bloat with macro-op fusion for RISC-V, 2016. <https://arxiv.org/abs/1607.02318>. [arXiv:1607.02318](https://arxiv.org/abs/1607.02318).
- [CFG⁺24] H. Cheng, G. Fotiadis, J. Großschädl, D. Page, T.H. Pham, and P.Y.A. Ryan. RISC-V instruction set extensions for multi-precision integer arithmetic. In *Design Automation Conference (DAC)*, pages 329:1–329:6, 2024. <https://doi.org/10.1145/3649329.3657347>.
- [CGM⁺24] H. Cheng, J. Großschädl, B. Marshall, D. Page, and M.-J.O. Saarinen. SoK: Instruction set extensions for cryptographers. Cryptology ePrint Archive, Paper 2024/1323, 2024. <https://eprint.iacr.org/2024/1323>.
- [CLM⁺18] W. Castryck, T. Lange, C. Martindale, L. Panny, and J. Renes. CSIDH: An efficient post-quantum commutative group action. In *Advances in Cryptology (ASIACRYPT)*, LNCS 11274, pages 395–427. Springer-Verlag, 2018. https://doi.org/10.1007/978-3-030-03332-3_15.
- [Com90a] P.G. Comba. Exponentiation cryptosystems on the IBM PC. *IBM Systems Journal*, 29(4):526–538, 1990. <https://doi.org/10.1147/sj.294.0526>.
- [Com90b] P.G. Comba. Exponentiation cryptosystems on the IBM PC. *IBM Systems Journal*, 29(4):526–538, December 1990.
- [EJ17] Nadia El Mrabet and Marc Joye. *Guide to Pairing-Based Cryptography*. Chapman & Hall/CRC, 2017.
- [EPG⁺19] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala. Simple high-level code for cryptographic arithmetic - with proofs, without compromises. In *IEEE Symposium on Security and Privacy (SP)*, pages 1202–1219. IEEE, 2019. <https://doi.org/10.1109/SP.2019.00005>.
- [EPJ⁺24] A. Erbsen, J. Philipoom, D. Jamner, A. Lin, S. Gruetter, C. Pit-Claudel, and A. Chlipala. Foundational integration verification of a cryptographic server. *Proceedings of the ACM on Programming Languages*, 8:1704–1729, 2024. <https://doi.org/10.1145/3656446>.
- [FGR24] G. Fotiadis, J. Großschädl, and P.Y.A. Ryan. X2065: lightweight key exchange for the internet of things. In *Cyber-Physical System Security Workshop (CPSS)*, pages 43–52, 2024. <https://doi.org/10.1145/3626205.3659153>.
- [Gal12] Steven D. Galbraith. *Mathematics of Public Key Cryptography*. Cambridge University Press, 2012.

- [GPW⁺04] N. Gura, A. Patel, A.S. Wander, H. Eberle, and S. Chang Shantz. Comparing elliptic curve cryptography and RSA on 8-bit CPUs. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 3156, pages 119–132. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-28632-5_9.
- [GSD⁺24] Q. Gao, K. Sun, J. Dong, F. Zheng, J. Lin, Y. Ren, and Z. Liu. V-Curve25519: Efficient implementation of Curve25519 on RISC-V architecture. In *Information Security and Cryptology (Inscrypt)*, LNCS 14527, pages 130–149. Springer-Verlag, 2024. https://doi.org/10.1007/978-981-97-0945-8_8.
- [Ham15] M. Hamburg. Ed448-Goldilocks, a new elliptic curve. Cryptology ePrint Archive, Paper 2015/625, 2015. <https://eprint.iacr.org/2015/625>.
- [HCP⁺16] M.D. Hill, D. Christie, D. Patterson, J.J. Yi, D. Chiou, and R. Sendag. Proprietary versus open instruction sets. *IEEE Micro*, 36(4):58–68, 2016. <https://doi.org/10.1109/MM.2016.61>.
- [HMOV04] D.R. Hankerson, A.J. Menezes, and S.A. Vanstone. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, 2004. <https://doi.org/10.1007/b97644>.
- [HS15] M. Hutter and P. Schwabe. Multiprecision multiplication on AVR revisited. *Journal of Cryptographic Engineering (JCEN)*, 5(3):201–214, 2015. <https://doi.org/10.1007/s13389-015-0093-2>.
- [JAC⁺20] D. Jao, R. Azarderakhsh, M. Campagna, C. Costello, L. De Feo, B. Hess, A. Hutchinson, A. Jalali, K. Karabina, B. Koziel, B. LaMacchia, P. Longa, M. Naehrig, G. Pereira, J. Renes, V. Soukharev, and D. Urbanik. Supersingular Isogeny Key Encapsulation. Technical report, 2020. <https://sike.org/files/SIDH-spec.pdf>.
- [MMP⁺23] L. Maino, C. Martindale, L. Panny, G. Pope, and B. Wesolowski. A direct key recovery attack on SIDH. In *Advances in Cryptology (EUROCRYPT)*, LNCS 14008, pages 423–447. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-30589-4_16.
- [MOV96] A.J. Menezes, P.C. van Oorschot, and S.A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. <https://cacr.uwaterloo.ca/hac>.
- [Nat99] National Institute of Standards and Technology (NIST). Recommended Elliptic Curves for Federal Government Use. White paper, 1999. <https://csrc.nist.gov/groups/ST/toolkit/documents/dss/NISTReCur.pdf>.
- [Nat25] National Institute of Standards and Technology (NIST). Considerations for achieving cryptographic agility: Strategies and practices. Cybersecurity White Paper (CSWP) 39 (2nd public draft), 2025. <https://doi.org/10.6028/NIST.CSWP.39.2pd>.
- [NL15] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 7539, 2015. <https://datatracker.ietf.org/doc/html/rfc7539>.
- [PPJ⁺22] C. Pit-Claudel, J. Philipoom, D. Jamner, A. Erbsen, and A. Chlipala. Relational compilation for performance-critical applications: extensible proof-producing translation of functional models into low-level code. In *Programming Language Design and Implementation (PLDI)*, pages 918–933. ACM, 2022. <https://doi.org/10.1145/3519939.3523706>.
- [PW17] D. Patterson and A. Waterman. *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017. <http://riscvbook.com>.

- [RIS24a] RISC-V. The RISC-V instruction set manual volume I: Unprivileged architecture. Ratified ISA Release 20240411, RISC-V International, 2024. <https://github.com/riscv/riscv-isa-manual/releases/download/20240411/unpriv-isa-asciidoc.pdf>.
- [RIS24b] RISC-V. The RISC-V instruction set manual volume II: Privileged architecture. Ratified ISA Release 20240411, RISC-V International, 2024. <https://github.com/riscv/riscv-isa-manual/releases/download/20240411/priv-isa-asciidoc.pdf>.
- [Rob23] D. Robert. Breaking SIDH in polynomial time. In *Advances in Cryptology (EUROCRYPT)*, LNCS 14008, pages 423–447. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-30589-4_17.
- [Sco15] M. Scott. Ed3363 (HighFive) – an alternative elliptic curve. Cryptology ePrint Archive, Paper 2015/991, 2015. <https://eprint.iacr.org/2015/991>.
- [Sco24] M. Scott. Elliptic curve cryptography for the masses: Simple and fast finite field arithmetic. Cryptology ePrint Archive, Paper 2024/779, 2024. <https://eprint.iacr.org/2024/779>.
- [SPJR22] S. Singh, A. Perais, A. Jimborean, and A. Ros. Exploring instruction fusion opportunities in general purpose processors. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 199–212, 2022. <https://doi.org/10.1109/MICRO56248.2022.00026>.
- [Sto19] K. Stoffelen. Efficient cryptography on the RISC-V architecture. In *Progress in Cryptology (LATINCRYPT)*, LNCS 11774, pages 323–340. Springer-Verlag, 2019. https://doi.org/10.1007/978-3-030-30530-7_16.
- [Vai17] L. Vaillant. The design of Poly1305. Blog post, 2017. <https://loup-vaillant.fr/tutorials/poly1305-design>.
- [vdB20] S.H.M. van den Berg. RISC-V implementation of the NaCl-library. MSc thesis, Eindhoven University of Technology, 2020. https://pure.tue.nl/ws/portalfiles/portal/169647601/Berg_S._ES_CSE.pdf.
- [Wat16] A. Waterman. *Design of the RISC-V Instruction Set Architecture*. PhD thesis, University of California at Berkeley, 2016. <http://www.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-1.html>.
- [ZB19] F. Zaruba and L. Benini. The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(11):2629–2640, 2019. <https://doi.org/10.1109/TVLSI.2019.2926114>.